

Clase 035 — IDS/IPS con Snort y Suricata

Parte: 1 — Redes y seguridad de redes · Fuente: *Applied NSM, C. Sanders & J. Smith; docs de Snort y Suricata* 🕒 Duración estimada: 140 min · Nivel: Intermedio

Objetivo

Desplegar y operar sistemas de detección/prevencción de intrusiones basados en firmas con **Snort** y **Suricata**: entender la diferencia entre IDS e IPS, escribir y afinar reglas, procesar tráfico en vivo y desde pcap, e interpretar las alertas resultantes. Es el pilar de la detección basada en firmas dentro de un SOC.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Diferenciar** IDS de IPS y sus modos de despliegue (inline, tap, span).
2. **Instalar** y configurar Suricata/Snort con conjuntos de reglas comunitarios.
3. **Escribir** reglas propias con cabecera y opciones (content, pcre, flow).
4. **Analizar** tráfico desde un archivo pcap y en vivo.
5. **Interpretar** alertas y reducir falsos positivos mediante afinado.
6. **Integrar** la salida (EVE JSON) con herramientas de análisis.

Temas

#	Tema	Por qué importa
1	IDS vs. IPS; modos de despliegue	Detectar vs. bloquear
2	Arquitectura de Suricata (multihilo)	Rendimiento en redes rápidas
3	Conjuntos de reglas (ET Open, Talos)	No reinventar firmas
4	Anatomía de una regla	Escribir detección propia
5	Opciones: content, pcre, flow, flowbits	Precisión de la firma
6	Salida EVE JSON y logs	Integración con SIEM
7	Afinado y falsos positivos	Detección usable en producción

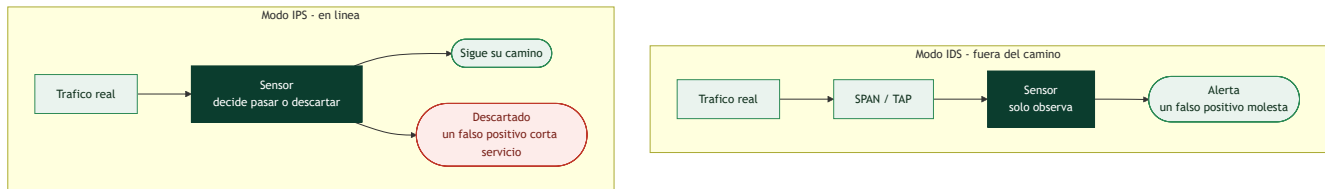
Explicación en profundidad

Detectar o bloquear: la decisión que define el despliegue

Un **IDS** observa una copia del tráfico y avisa; un **IPS** se sitúa en el camino del tráfico y puede descartarlo. La misma herramienta —Suricata lo es— hace las dos cosas según cómo se despliegue, y la elección tiene consecuencias que van mucho más allá de lo técnico. En modo IDS, sobre un SPAN o un TAP, un falso positivo genera una alerta molesta; en modo IPS, en línea, un falso positivo **corta tráfico legítimo** y

produce una incidencia de negocio. Y como el IPS está en el camino, si se cae o se satura se convierte en un punto único de fallo para toda la red que protege.

Por eso el patrón habitual en organizaciones sensatas es empezar en modo IDS, afinar las reglas hasta que el ruido sea manejable, y solo entonces pasar en línea las firmas de las que se tiene confianza alta. El bloqueo se gana; no se activa el primer día.



La anatomía de una regla, leída como una frase

Una firma de Suricata o Snort se lee de izquierda a derecha como una oración con dos partes. La **cabecera** dice qué hacer y sobre qué tráfico: acción (`alert` , `drop` , `pass`), protocolo, dirección de origen con puerto, el operador de dirección (`→`) y destino con puerto. Las **opciones**, entre paréntesis, refinan la condición y aportan los metadatos.

```
alert http $EXTERNAL_NET any → $HOME_NET any ( \
  msg:"Descarga de ejecutable desde host recién registrado"; \
  flow:established,to_client; \
  http.content_type; content:"application/x-msdownload"; \
  classtype:trojan-activity; sid:1000010; rev:1; )
```

Las opciones que más determinan la calidad de una firma son cuatro. `flow` acota la dirección y el estado de la conexión, y evita que la regla dispare sobre tráfico suelto o en el sentido equivocado. Los **buffers de protocolo** — `http.uri` , `http.user_agent` , `tls.sni` , `dns.query` — restringen la búsqueda al campo correcto en lugar de rastrear todo el paquete, lo que multiplica la precisión y el rendimiento. `content` busca una cadena literal y es barato; `pcrc` aplica una expresión regular y es caro, así que la práctica correcta es poner primero un `content` que descarte la mayoría del tráfico y solo después el `pcrc` . Y `flowbits` permite **encadenar estado entre paquetes**: marcar en un paquete y comprobar la marca en otro, que es como se detectan secuencias en lugar de eventos aislados. El `sid` identifica la regla —el rango local empieza en 1000000— y el `rev` su versión.

El problema real no es escribir reglas: es el ruido

Un despliegue con `ET Open` completo genera decenas de miles de alertas diarias en una red mediana, y un SOC que recibe eso deja de mirar las alertas en una semana. El trabajo de verdad es el **afinado**: desactivar categorías que no aplican a tu entorno, crear excepciones por origen para escáneres de vulnerabilidades y sistemas de monitorización propios, ajustar `threshold` y `suppress` para limitar repeticiones, y medir cuántas alertas se investigan de verdad. Una regla que nadie mira nunca es peor que no tener la regla, porque da una sensación de cobertura que no existe.

Conviene también conocer los límites del método. La detección por firma solo reconoce lo que alguien ya describió: no ve un ataque nuevo, y **no ve dentro del tráfico cifrado**. Con TLS omnipresente, lo que queda visible son los metadatos —el `SNI` , el certificado, los tamaños y tiempos, la huella JA3—, y por eso este bloque desemboca de forma natural en el análisis de metadatos de las clases 043 a 045. La salida **EVE**

JSON de Suricata es la pieza que conecta todo eso con un SIEM: un evento estructurado por línea, con alertas y también con registros de transacción HTTP, DNS y TLS.

Definiciones y características

- **IDS:** detecta y alerta sobre tráfico sospechoso pero no lo bloquea; se despliega fuera de línea (tap/span).
- **IPS:** se coloca en línea (inline) y puede **descartar** el tráfico que coincide con una firma; requiere alta fiabilidad para no cortar tráfico legítimo.
- **Regla/firma:** patrón que describe tráfico malicioso: una cabecera (acción, protocolo, IP/puertos, dirección) y opciones (qué buscar y cómo alertar).
- **content** : cadena o bytes a buscar en el payload; núcleo de la mayoría de firmas.
- **flow** : restringe la regla a una dirección/estado de la conexión (`to_server`, `established`).
- **EVE JSON:** formato de salida estructurado de Suricata que registra alertas, flujos, DNS, TLS, HTTP, etc.

Glosario

Término	Definición concisa
IDS	Detecta y alerta observando una copia del tráfico
IPS	Se sitúa en línea y puede descartar el tráfico
Modo en línea	Despliegue en el camino del tráfico; un fallo corta el servicio
Firma / regla	Descripción de un patrón de tráfico considerado malicioso
Cabecera de regla	Acción, protocolo, origen, dirección y destino
<code>flow</code>	Acota dirección y estado de la conexión en la que aplica la regla
Buffer de protocolo	Campo concreto donde buscar (<code>http.uri</code> , <code>tls.sni</code> , <code>dns.query</code>)
<code>content</code>	Búsqueda de cadena literal; rápida
<code>pcre</code>	Expresión regular; precisa pero costosa, se pone tras un <code>content</code>
<code>flowbits</code>	Marca de estado que encadena condiciones entre paquetes
<code>sid</code> / <code>rev</code>	Identificador y versión de la regla; el rango local empieza en 1000000
ET Open	Conjunto de reglas abierto de Emerging Threats
Afinado (<i>tuning</i>)	Reducir el ruido para que las alertas sean investigables
<code>threshold</code> / <code>suppress</code>	Limitar repeticiones y silenciar orígenes conocidos
EVE JSON	Salida estructurada de Suricata: alertas y transacciones, una por línea
JA3	Huella del cliente TLS; identifica software sin descifrar el tráfico

Herramientas y preparación

- **Suricata 7.x:** `sudo apt install suricata` . Actualiza reglas con `suricata-update` .
- **Snort 3.x** (opcional para comparar): `sudo apt install snort` o compilación oficial.

- Reglas comunitarias: **Emerging Threats Open** (`suricata-update` las trae).
- Herramientas de lectura: `jq` para el EVE JSON, `tail`, un pcap de prueba.

⚠ **Nota ética:** para probar firmas generarás tráfico que simula ataques (escaneos, patrones maliciosos). Hazlo **solo** en tu laboratorio aislado. Un IPS mal configurado en producción puede cortar servicio; prueba primero en modo IDS.

Laboratorio guiado

1. Descarga reglas y verifica configuración:

```
bash sudo suricata-update sudo suricata -T -c /etc/suricata/suricata.yaml # test de config
```

2. Analiza un pcap con Suricata:

```
bash sudo suricata -r /tmp/lab027.pcapng -l ./salida/ jq 'select(.event_type=="alert") | .alert.signature' ./salida/eve.json
```

3. Modo IDS en vivo sobre una interfaz:

```
bash sudo suricata -i eth0 -l /var/log/suricata/ sudo tail -f /var/log/suricata/fast.log
```

4. Escribe una regla propia en `/etc/suricata/rules/local.rules`:

```
text alert icmp any any → $HOME_NET any (msg:"ICMP ping detectado en laboratorio"; itype:8; sid:1000001; rev:1;) alert http any any → any any (msg:"Posible user-agent de escaneo"; flow:to_server,established; http.user_agent; content:"Nmap"; nocase; sid:1000002; rev:1;)
```

Añade `local.rules` a `rule-files` en el YAML y recarga. 5. **Dispara la regla:** desde otra VM `ping <IP-de-un-host-en-HOME_NET>` (una IP concreta de tu red de laboratorio; no uses `$HOME`, que la shell expande a tu directorio personal) y un `curl -A "Nmap NSE" http:// ...`; observa las alertas en `fast.log`. 6.

Compara con Snort 3 (opcional):

```
bash snort -c /etc/snort/snort.lua -r /tmp/lab027.pcapng -A alert_fast
```

7. **Afina un falso positivo:** identifica una firma ruidosa y desactívala con `suricata-update --disable-conf` o un umbral (`threshold.config`).

Ejercicios

1. Escribe una regla que alerte sobre intentos de conexión a un puerto de administración (p. ej. 3389).
2. Crea una firma con `pcre` que detecte una cadena en URL con expresión regular.
3. Usa `flowbits` para correlacionar dos eventos de una misma sesión.
4. Procesa un pcap con tráfico DNS y extrae con `jq` todas las consultas registradas por Suricata.
5. Configura un umbral (`threshold`) para limitar alertas repetidas de una misma firma.
6. Compara el rendimiento (paquetes/s) de Suricata multihilo frente a Snort en el mismo pcap.

Reto verificable

Despliega Suricata en modo IDS en tu laboratorio, escribe una regla propia que detecte un patrón de ataque concreto (p. ej. una petición HTTP a `/admin` con cierto user-agent), genera el tráfico que la dispara y entrega: la regla, la línea de `fast.log` correspondiente y el objeto de alerta del `eve.json` extraído con `jq`.

Criterio de aceptación: la regla tiene `sid` propio válido, la alerta aparece con el `msg` exacto al generar el tráfico, y no se dispara con tráfico legítimo similar (bajo falso positivo).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
"Signature has no sid"	Falta <code>sid:</code> en la regla; añade un SID único ≥ 1000000 para reglas locales
La regla no dispara	<code>\$HOME_NET</code> mal definido o <code>flow</code> incorrecto; revisa variables en el YAML y la dirección
Suricata no captura en vivo	Interfaz equivocada o sin permisos; verifica <code>-i</code> y ejecuta con privilegios
Miles de alertas ruidosas	Reglas demasiado genéricas activas; afina con <code>suricata-update</code> y umbrales
IPS corta tráfico legítimo	Firma con falso positivo en modo inline; valida siempre primero en modo IDS

Preguntas frecuentes

? ¿Snort o Suricata? Suricata es multihilo (mejor en redes rápidas) y trae EVE JSON nativo. Snort 3 también es moderno. Ambos comparten en gran medida el formato de reglas; aprende Suricata como base.

? ¿La detección por firmas detecta amenazas nuevas? No las desconocidas (zero-day) sin firma. Por eso se combina con detección por anomalías y con NSM (clases 043–045).

? ¿Qué es `HOME_NET`? La variable que define tu red a proteger. Firmas y direcciones ($\rightarrow$ `$HOME_NET`) dependen de configurarla bien en el YAML.

? ¿Puedo usar las mismas reglas en Snort y Suricata? En gran medida sí, comparten sintaxis, pero hay opciones específicas de cada motor. Verifica compatibilidad al portar reglas.

Referencias

- Suricata Documentation. <https://docs.suricata.io/>
- Snort 3 User Guide. <https://docs.snort.org/>
- Emerging Threats Open Ruleset. <https://rules.emergingthreats.net/>
- Sanders, C. & Smith, J. *Applied Network Security Monitoring*. Syngress.

Material descargable

-  [Guía en PDF](#) — versión imprimible de esta clase.
-  [Presentación \(PPTX\)](#) — deck para proyectar en clase.

Clase anterior

Clase 034 — Firewalls: tipos, iptables y nftables

Siguiente clase

Clase 036 — VPN y túneles: IPsec, WireGuard y OpenVPN